

2520090148_SAINATH

1. Database Commands

show dbs

use studentDB

db

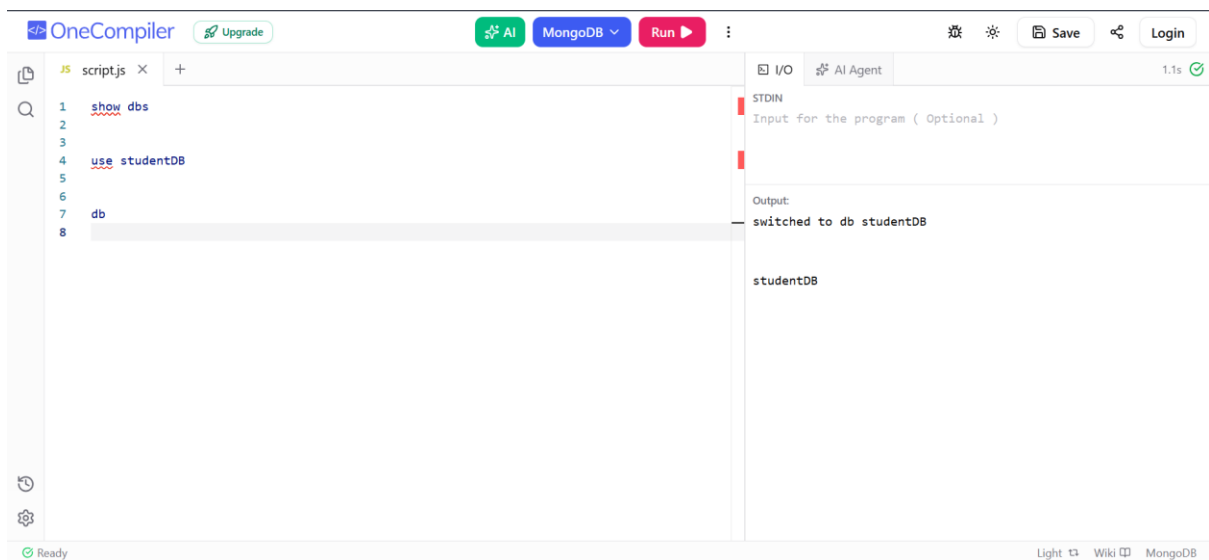
db.dropDatabase()

show dbs – Show all databases

use studentDB – Create/select database

db – Show current database

db.dropDatabase() – Delete current database



The screenshot shows the OneCompiler web interface. The top bar includes the OneCompiler logo, an 'Upgrade' button, and tabs for 'AI', 'MongoDB', and 'Run'. The main editor area displays a JavaScript file named 'script.js' with the following code:

```
1 show dbs
2
3
4 use studentDB
5
6
7 db
8
```

The right sidebar shows the 'I/O' tab with 'AI Agent' and '1.1s' status. The 'STDIN' section is empty. The 'Output' section shows the result of the commands:

```
switched to db studentDB

studentDB
```

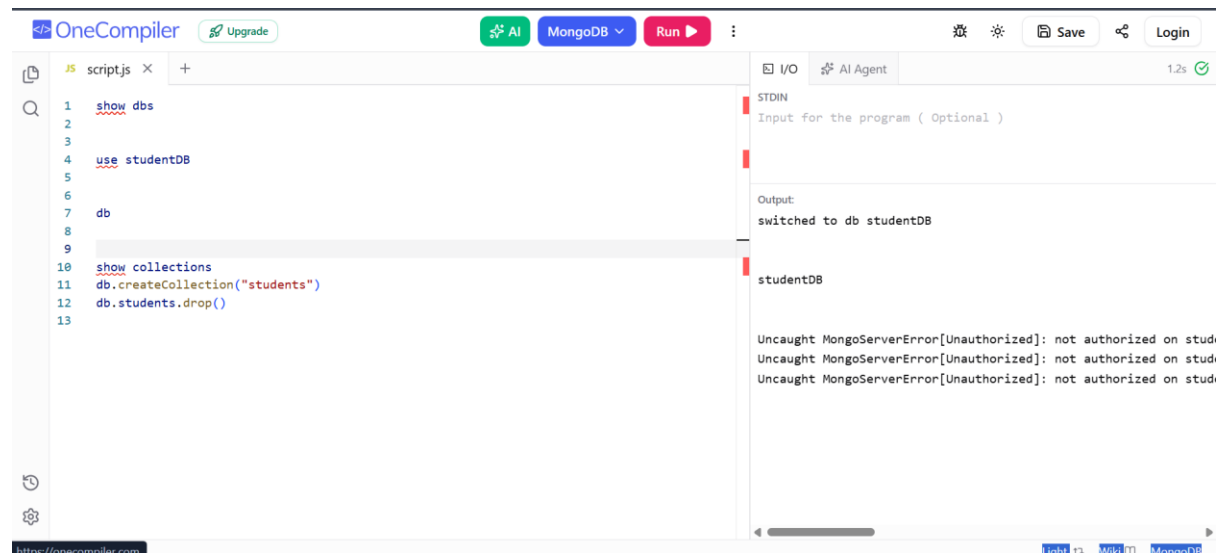
The bottom status bar indicates 'Ready' and includes links for 'Light', 'Wiki', and 'MongoDB'.

2. Collection Commands

show collections

```
db.createCollection("students")
```

```
db.students.drop()
```



The screenshot shows the OneCompiler web IDE interface. The left pane contains a JavaScript file named 'script.js' with the following code:

```
1 show dbs
2
3
4 use studentDB
5
6
7 db
8
9
10 show collections
11 db.createCollection("students")
12 db.students.drop()
13
```

The right pane shows the output of the commands. The 'Output' section displays:

```
switched to db studentDB
studentDB
```

Below the output, there are three error messages:

```
Uncaught MongoServerError[Unauthorized]: not authorized on stud
Uncaught MongoServerError[Unauthorized]: not authorized on stud
Uncaught MongoServerError[Unauthorized]: not authorized on stud
```

The top of the interface includes a navigation bar with 'OneCompiler', 'Upgrade', 'AI', 'MongoDB', 'Run', 'Save', and 'Login' buttons. The bottom status bar shows '1.2s' and a green checkmark.

3. Sample Student Data

```
db.students.insertMany([
  { rollNo: 101, name: "John", age: 20, department: "CSE", marks: 85, city: "Hyderabad" },
  { rollNo: 102, name: "Alice", age: 21, department: "CSE", marks: 90, city: "Vijayawada" },
  { rollNo: 103, name: "Rahul", age: 20, department: "ECE", marks: 75, city: "Hyderabad" },
  { rollNo: 104, name: "Priya", age: 22, department: "IT", marks: 88, city: "Warangal" },
  { rollNo: 105, name: "Kiran", age: 21, department: "CSE", marks: 95, city: "Hyderabad" },
  { rollNo: 106, name: "Anu", age: 20, department: "IT", marks: 82, city: "Guntur" },
  { rollNo: 107, name: "Ravi", age: 23, department: "ECE", marks: 68, city: "Nellore" },
  { rollNo: 108, name: "Sita", age: 21, department: "CSE", marks: 91, city: "Hyderabad" },
  { rollNo: 109, name: "Arun", age: 22, department: "AI&DS", marks: 87, city: "Kurnool" },
  { rollNo: 110, name: "Meena", age: 20, department: "AI&DS", marks: 93, city: "Hyderabad" }
])
```

OneCompiler

Upgrade

AI

MongoDB

Run

Save

Login

script.js

+

12s

```
1
2
3
4 use studentDB
5
6
7 db
8
9
10 show collections
11 db.createCollection("students")
12 db.students.drop()
13 db.students.insertMany([
14   { rollNo: 101, name: "John", age: 20, department: "CSE", marks: 85, city: "Hyderabad" },
15   { rollNo: 102, name: "Alice", age: 21, department: "CSE", marks: 90, city: "Vijayawada" },
16   { rollNo: 103, name: "Rahul", age: 20, department: "ECE", marks: 75, city: "Hyderabad" },
17   { rollNo: 104, name: "Priya", age: 22, department: "IT", marks: 88, city: "Warangal" },
18   { rollNo: 105, name: "Kiran", age: 21, department: "CSE", marks: 95, city: "Hyderabad" },
19   { rollNo: 106, name: "Anu", age: 20, department: "IT", marks: 82, city: "Guntur" },
20   { rollNo: 107, name: "Ravi", age: 23, department: "ECE", marks: 68, city: "Nellore" },
21   { rollNo: 108, name: "Sita", age: 21, department: "CSE", marks: 91, city: "Hyderabad" },
22   { rollNo: 109, name: "Arun", age: 22, department: "AI&DS", marks: 87, city: "Kurnool" },
23   { rollNo: 110, name: "Meena", age: 20, department: "AI&DS", marks: 93, city: "Hyderabad" }
24 ])
25
26
```

STDIN

Input for the program (Optional)

Output:

upsertedIds: {},

insertedIds: {

'0': ObjectId('6a7ed14c29f6a237d58ce5af'),

'1': ObjectId('6a7ed14c29f6a237d58ce5b0'),

'2': ObjectId('6a7ed14c29f6a237d58ce5b1'),

'3': ObjectId('6a7ed14c29f6a237d58ce5b2'),

'4': ObjectId('6a7ed14c29f6a237d58ce5b3'),

'5': ObjectId('6a7ed14c29f6a237d58ce5b4'),

'6': ObjectId('6a7ed14c29f6a237d58ce5b5'),

'7': ObjectId('6a7ed14c29f6a237d58ce5b6'),

'8': ObjectId('6a7ed14c29f6a237d58ce5b7'),

'9': ObjectId('6a7ed14c29f6a237d58ce5b8')

}

Write Errors: []

Ready

Light

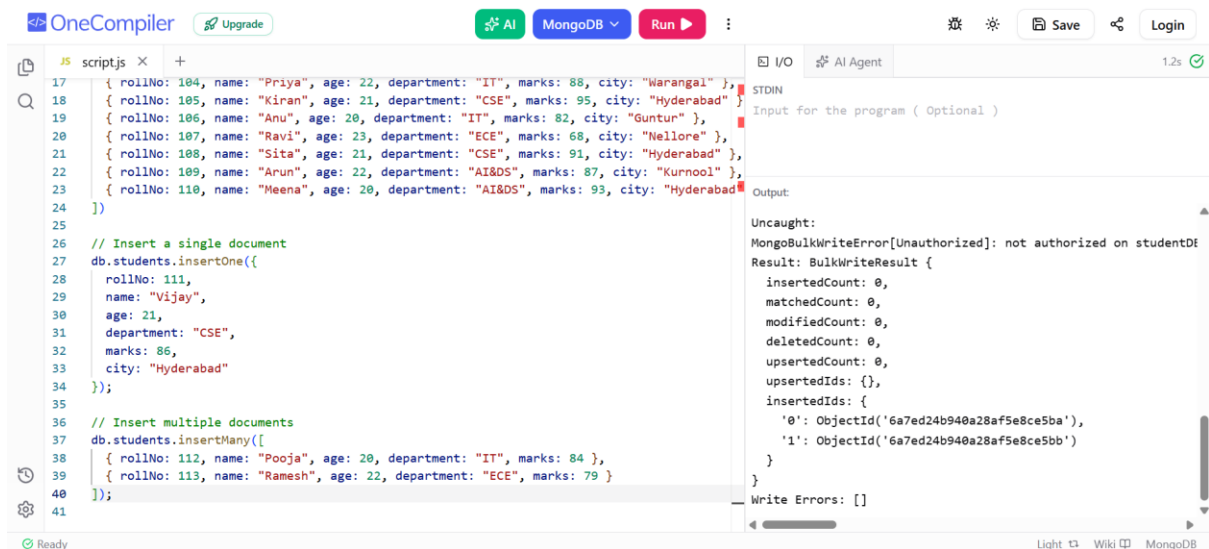
Wiki

MongoDB

4. CREATE – Insert Commands

```
db.students.insertOne({
  rollNo: 111,
  name: "Vijay",
  age: 21,
  department: "CSE",
  marks: 86,
  city: "Hyderabad"
})
```

```
db.students.insertMany([
  { rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 },
  { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }
])
```



The screenshot shows the OneCompiler web interface for MongoDB. The left pane contains a JavaScript script with the following code:

```
17 { rollNo: 104, name: "Priya", age: 22, department: "IT", marks: 88, city: "Warangal" },
18 { rollNo: 105, name: "Kiran", age: 21, department: "CSE", marks: 95, city: "Hyderabad" },
19 { rollNo: 106, name: "Anu", age: 20, department: "IT", marks: 82, city: "Guntur" },
20 { rollNo: 107, name: "Ravi", age: 23, department: "ECE", marks: 68, city: "Nellore" },
21 { rollNo: 108, name: "Sita", age: 21, department: "CSE", marks: 91, city: "Hyderabad" },
22 { rollNo: 109, name: "Arun", age: 22, department: "AI&DS", marks: 87, city: "Kurnool" },
23 { rollNo: 110, name: "Meena", age: 20, department: "AI&DS", marks: 93, city: "Hyderabad"
24 }
25
26 // Insert a single document
27 db.students.insertOne({
28   rollNo: 111,
29   name: "Vijay",
30   age: 21,
31   department: "CSE",
32   marks: 86,
33   city: "Hyderabad"
34 });
35
36 // Insert multiple documents
37 db.students.insertMany([
38   { rollNo: 112, name: "Pooja", age: 20, department: "IT", marks: 84 },
39   { rollNo: 113, name: "Ramesh", age: 22, department: "ECE", marks: 79 }
40 ]);
41
```

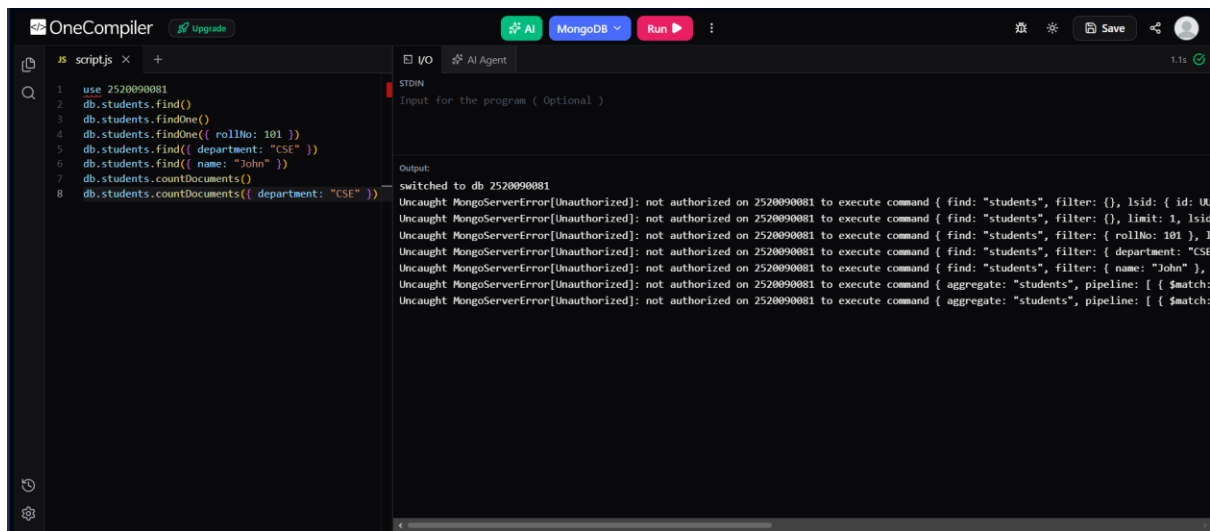
The right pane shows the output of the script execution. It displays an "Uncaught" error: `MongoBulkWriteError[Unauthorized]: not authorized on studentDI`. Below the error, it shows the `BulkWriteResult` object:

```
Result: BulkWriteResult {
  insertedCount: 0,
  matchedCount: 0,
  modifiedCount: 0,
  deletedCount: 0,
  upsertedCount: 0,
  upsertedIds: {},
  insertedIds: {
    '0': ObjectId('6a7ed24b940a28af5e8ce5ba'),
    '1': ObjectId('6a7ed24b940a28af5e8ce5bb')
  }
}
```

The bottom status bar indicates the application is "Ready" and shows the "MongoDB" database selected.

5. READ – Basic Find Commands

```
db.students.find()
db.students.findOne()
db.students.findOne({ rollNo: 101 })
db.students.find({ department: "CSE" })
db.students.find({ name: "John" })
db.students.countDocuments()
db.students.countDocuments({ department: "CSE" })
```



6. Comparison Operators

// Greater than

```
db.students.find({ marks: { $gt: 80 } })
```

// Greater than or equal

```
db.students.find({ marks: { $gte: 80 } })
```

// Less than

```
db.students.find({ marks: { $lt: 80 } })
```

// Less than or equal

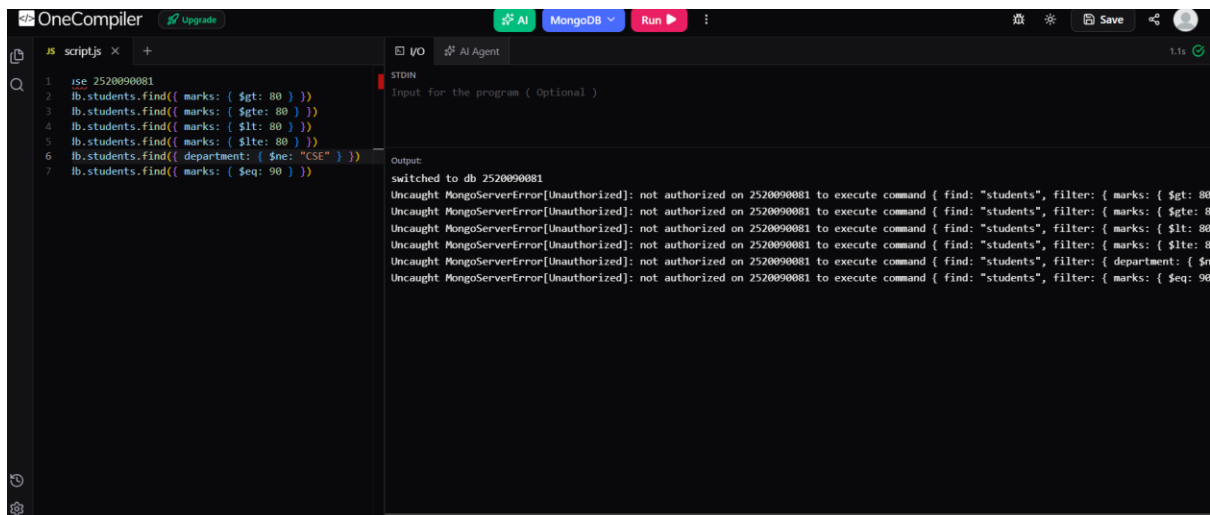
```
db.students.find({ marks: { $lte: 80 } })
```

// Not equal

```
db.students.find({ department: { $ne: "CSE" } })
```

// Equal

```
db.students.find({ marks: { $eq: 90 } })
```



7. Logical Operators

// AND

```
db.students.find({
  $and: [
    { department: "CSE" },
    { marks: { $gt: 85 } }
  ]
})
```

// OR

```
db.students.find({
  $or: [
    { department: "CSE" },
    { department: "IT" }
  ]
})
```

// NOT

```
db.students.find({
  marks: { $not: { $gt: 80 } }
})
```

// NOR

```
db.students.find({
  $nor: [
    { department: "CSE" },
    { marks: { $gt: 90 } }
  ]
})
```

```
1 use 2520090081
2 db.students.find(
3   $and: [
4     { department: "CSE" },
5     { marks: { $gt: 85 } }
6   ]
7 )
8 db.students.find(
9   $or: [
10    { department: "CSE" },
11    { department: "IT" }
12  ]
13 )
14 db.students.find(
15   marks: { $not: { $gt: 80 } }
16 )
17 db.students.find(
18   $nor: [
19     { department: "CSE" },
20     { marks: { $gt: 90 } }
21   ]
22 )
23 db.students.insertMany([
24   { name: "Alice", department: "CSE", marks: 92 },
25   { name: "Bob", department: "IT", marks: 88 },
26   { name: "Charlie", department: "ECE", marks: 75 },
27   { name: "David", department: "CSE", marks: 78 }
28 ])
```

Output:

```
switched to db 2520090081
Uncaught MongoServerError[Unauthorized]: not authorized on 2520090081 to execute command { find: "students", filter: { $and: [ { depar
Uncaught MongoServerError[Unauthorized]: not authorized on 2520090081 to execute command { find: "students", filter: { $or: [ { depar
Uncaught MongoServerError[Unauthorized]: not authorized on 2520090081 to execute command { find: "students", filter: { marks: { $not:
Uncaught MongoServerError[Unauthorized]: not authorized on 2520090081 to execute command { find: "students", filter: { $nor: [ { depar
Uncaught:
MongoBulkWriteError[Unauthorized]: not authorized on 2520090081 to execute command { insert: "students", documents: 4, ordered: true,
Result: BulkWriteResult {
  insertedCount: 0,
  matchedCount: 0,
  modifiedCount: 0,
  deletedCount: 0,
  upsertedCount: 0,
  upsertedIds: {},
  insertedIds: {
    '0': ObjectId('6a8d2d14769971165e8ce5af'),
    '1': ObjectId('6a8d2d14769971165e8ce5b0'),
    '2': ObjectId('6a8d2d14769971165e8ce5b1'),
    '3': ObjectId('6a8d2d14769971165e8ce5b2')
  }
}
```

8. IN and NIN Operators

```
db.students.find({
  department: { $in: ["CSE", "IT"] }
})
```

```
db.students.find({
  department: { $nin: ["CSE", "IT"] }
})
```

```
1 use 2520090081
2 db.students.find(
3   department: { $in: ["CSE", "IT"] }
4 )
5 db.students.find(
6   department: { $nin: ["CSE", "IT"] }
7 )
```

Output:

```
switched to db 2520090081
Uncaught MongoServerError[Unauthorized]: not authorized on 2520090081 to execute command { find: "students", filter: { department: { $
Uncaught MongoServerError[Unauthorized]: not authorized on 2520090081 to execute command { find: "students", filter: { department: { $
```

9. UPDATE Commands

```
// Update one
db.students.updateOne(
  { rollNo: 101 },
  { $set: { marks: 92 } }
)
```

```
// Update multiple
db.students.updateMany(
```

```

    { department: "CSE" },
    { $set: { status: "Active" } }
  )

// Increment
db.students.updateOne(
  { rollNo: 101 },
  { $inc: { marks: 5 } }
)

// Decrement
db.students.updateOne(
  { rollNo: 101 },
  { $inc: { marks: -2 } }
)

// Rename a field
db.students.updateMany(
  {},
  { $rename: { "city": "location" } }
)

// Add field
db.students.updateOne(
  { rollNo: 101 },
  { $set: { attendance: 90 } }
)

// Remove a field
db.students.updateOne(
  { rollNo: 101 },
  { $unset: { attendance: "" } }
)

```

The screenshot shows the OneCompiler IDE interface. On the left, a code editor displays a JavaScript script for connecting to a MongoDB database and performing various update operations. The script includes comments for each operation: incrementing marks, decrementing marks, renaming a field, adding a field, and removing a field. The code is as follows:

```

1 use 2520090081
2 db.students.updateOne(
3   { rollNo: 101 },
4   { $set: { marks: 92 } }
5 )
6 db.students.updateMany(
7   { department: "CSE" },
8   { $set: { status: "Active" } }
9 )
10 db.students.updateOne(
11   { rollNo: 101 },
12   { $inc: { marks: 5 } }
13 )
14 db.students.updateOne(
15   { rollNo: 101 },
16   { $inc: { marks: -2 } }
17 )
18 db.students.updateMany(
19   {},
20   { $rename: { "city": "location" } }
21 )
22 db.students.updateOne(
23   { rollNo: 101 },
24   { $set: { attendance: 90 } }
25 )
26 db.students.updateOne(
27   { rollNo: 101 },
28   { $unset: { attendance: "" } }
29 )

```

On the right, the 'Output' pane shows the execution results. It starts with 'switched to db 2520090081' and then displays a series of 'Uncaught MongoServerError[Unauthorized]: not authorized on 2520090081 to execute command { update: "students", updates: [{ q: { rollNo: 101 }, u: { \$set: { marks: 92 } } }] }' for each update operation. This indicates that the user is not authorized to perform these updates on the specified database.

xfzzg